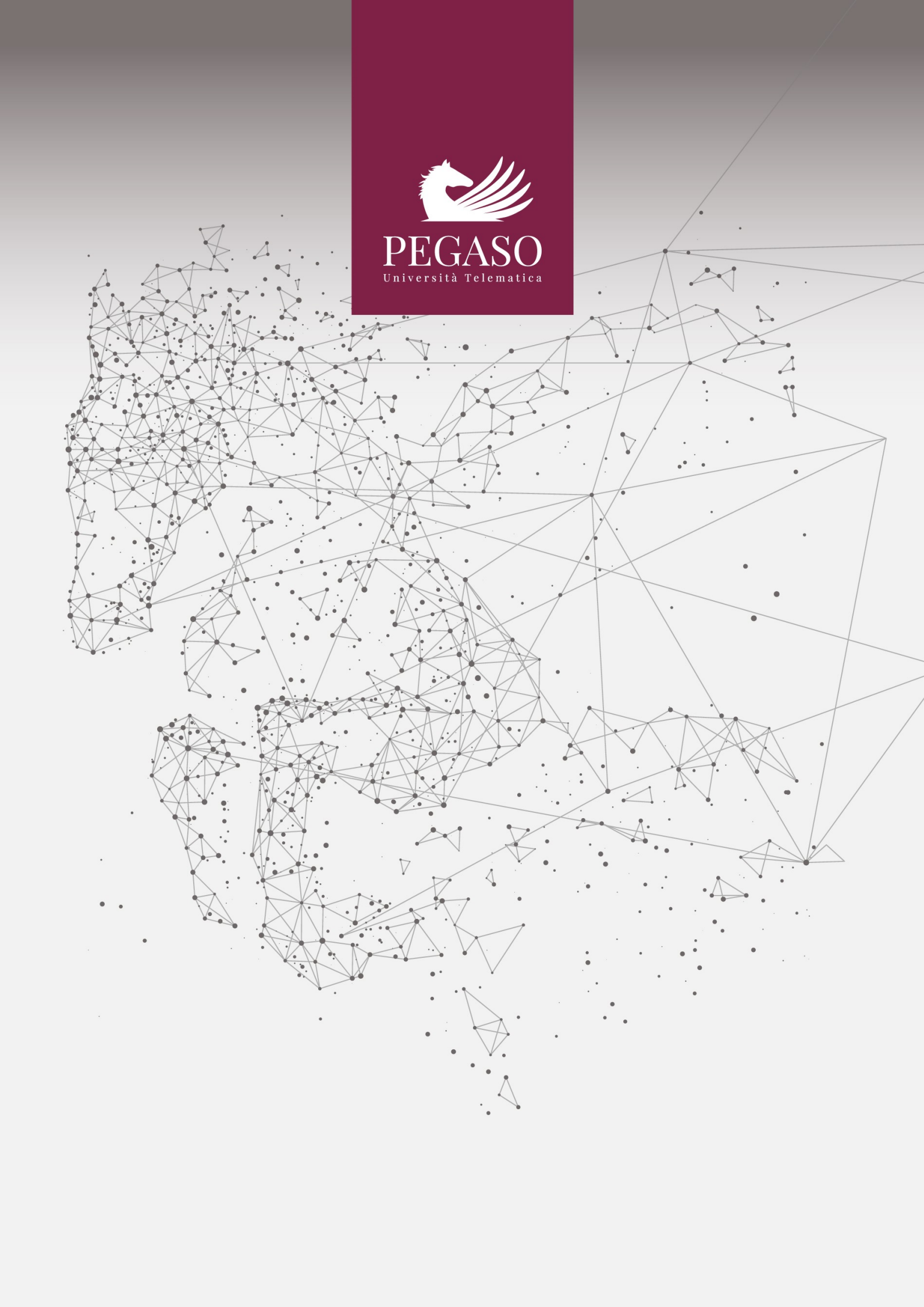

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. MODELLO MESSAGE-QUEUING 3

2. ARCHITETTURA DI UN SISTEMA DI MESSAGE-QUEUING 6

3. BROKER DI MESSAGGI 8

BIBLIOGRAFIA 11

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Modello message-queuing

I sistemi message-queuing, o semplicemente middleware orientato ai messaggi (MOM), sono una categoria significativa di servizi middleware orientati ai messaggi. I sistemi message-queuing forniscono un supporto completo per la comunicazione asincrona persistente. L'essenza di questi sistemi è che forniscono l'archiviazione dei messaggi per il periodo intermedio senza richiedere al mittente o al destinatario di essere attivi durante la trasmissione. Una distinzione fondamentale rispetto a socket e MPI è che i sistemi message-queuing sono in genere progettati per supportare trasferimenti di messaggi che possono richiedere minuti anziché millisecondi o secondi.

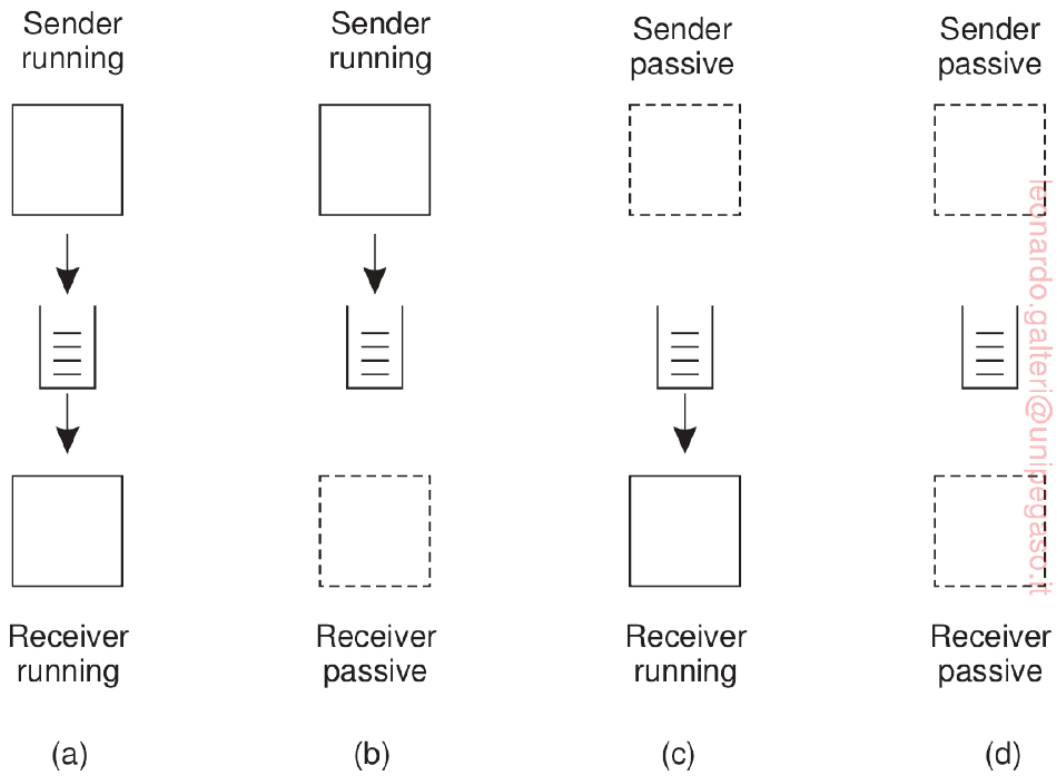
Un sistema message-queuing si basa sull'idea che le applicazioni comunichino inserendo messaggi in code specifiche. Anche se il server di destinazione è inattivo al momento dell'invio del messaggio, questi vengono inoltrati tramite una serie di server di comunicazione e infine recapitati. Nella pratica, la maggior parte dei server di comunicazione sono collegati direttamente. In altre parole, un messaggio viene in genere inviato direttamente al server di destinazione. Ogni applicazione ha, in teoria, una propria coda privata a cui altre applicazioni possono inviare messaggi. Una coda può essere letta solo dall'applicazione a cui è associata, ma più applicazioni possono condividere una singola coda.

Un aspetto cruciale dei sistemi message-queuing è che ad un mittente viene in genere data solo la certezza che il suo messaggio verrà aggiunto alla coda del destinatario. Non ci sono garanzie su quando o anche se il messaggio verrà letto; questo dipende interamente dalle azioni del destinatario. Questa semantica consente una comunicazione senza accoppiamento temporale.

Pertanto, non è necessario che il destinatario sia in esecuzione quando un messaggio viene inviato alla sua coda. Allo stesso modo, non è necessario che il mittente sia in esecuzione nel momento in cui il messaggio viene ricevuto dal destinatario. Mittente e destinatario possono operare in modo completamente indipendente l'uno dall'altro. Una volta inserito in una coda, un messaggio rimarrà lì fino a quando non viene rimosso, indipendentemente dal fatto che il mittente o il destinatario sia in esecuzione. Ciò si traduce in quattro combinazioni di modalità di esecuzione per il mittente e il destinatario, come illustrato nella Figura 1.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



leonardo.galteri@unipegaso.it

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Nella Figura 1(a), sia il mittente che il destinatario sono in esecuzione per tutta la durata della trasmissione di un messaggio. Nella figura 1(b), solo il mittente è attivo, mentre il destinatario si trova in uno stato in cui il recapito dei messaggi è impossibile. Tuttavia, il mittente può ancora trasmettere messaggi. La figura 1(c) illustra la combinazione di un mittente passivo e di un ricevitore in esecuzione. In questo caso, il destinatario è in grado di leggere i messaggi che gli sono stati inviati, ma non è necessario che anche i mittenti siano in esecuzione. La figura 1(d) illustra il sistema che memorizza (ed eventualmente trasmette) i messaggi anche quando sia il mittente che il destinatario sono passivi. Si potrebbe affermare che il sistema message-queuing fornisce messaggi persistenti solo se questa configurazione finale è supportata.

I messaggi possono teoricamente contenere qualsiasi tipo di dati. L'unico aspetto che conta dal punto di vista del middleware è che i messaggi siano indirizzati correttamente.

In pratica, l'indirizzamento viene eseguito fornendo un nome univoco a livello di sistema per la coda di destinazione. In alcuni casi, la dimensione dei messaggi potrebbe essere limitata; tuttavia, il sistema

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sottostante può frammentare e riassemblare messaggi di grandi dimensioni in modo completamente trasparente per le applicazioni.

Come illustrato nella Figura 2, come risultato di questo metodo, l'interfaccia fondamentale fornita alle applicazioni può essere estremamente semplice.

Operation	Description
`put`	Append a message to a specified queue
`get`	Block until the specified queue is nonempty, and remove the first message
`poll`	Check a specified queue for messages, and remove the first. Never block
`notify`	Install a handler to be called when a message is put into the specified queue

Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Un mittente richiama l'operazione *put* per passare un messaggio al sistema sottostante per l'aggiunta alla coda specificata. Questa chiamata non è bloccante. In secondo luogo, *get* è un'operazione di blocco che consente a un processo autorizzato di rimuovere il messaggio in sospeso più lungo dalla coda specificata. Il processo viene interrotto solo se non sono presenti elementi nella coda. Varianti di questo metodo consentono di cercare un particolare messaggio nella coda, ad esempio utilizzando una priorità o una corrispondenza di modello. L'operazione *poll* fornisce la variante che non è bloccante, a differenza dell'operazione precedente.

Se non sono presenti messaggi nella coda o non è stato possibile individuare un messaggio specifico, il processo di chiamata continua.

Inoltre, la maggior parte dei sistemi message-queuing consente ad un processo di installare un handler come funzione di callback che viene richiamata automaticamente ogni volta che un messaggio viene aggiunto alla coda. Le callback possono anche essere utilizzate per avviare automaticamente un processo che recupera i messaggi dalla coda se nessun processo è attualmente in esecuzione. Questa strategia viene spesso implementata da un demone sul lato ricevente che monitora continuamente la coda per i messaggi in arrivo e risponde di conseguenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Architettura di un sistema di message-queuing

Esaminiamo ora un sistema generale di message-queuing in modo più dettagliato. Prima di tutto, le code sono gestite dai gestori delle code (*queue managers*). Un processo separato o una libreria collegata all'applicazione implementa un gestore code. Come regola generale, un'applicazione può inviare messaggi solo a una coda locale. Allo stesso modo, è possibile recuperare un messaggio solo estraendolo da una coda locale. Di conseguenza, se un gestore di code QM_A che gestisce le code per un'applicazione **A** viene eseguito come processo separato, i processi QM_A e **A** vengono in genere collocati sullo stesso computer o, nel peggiore dei casi, sulla stessa LAN. Si noti che non è possibile fare riferimento a un sistema di messaggistica asincrona persistente se tutti i gestori code sono integrati nelle rispettive applicazioni.

Se le applicazioni possono inserire messaggi solo in code locali, ogni messaggio deve contenere informazioni sulla destinazione. È responsabilità del gestore della coda assicurarsi che un messaggio raggiunga il destinatario previsto. Ciò solleva una serie di preoccupazioni.

È necessario considerare inizialmente come viene gestita la coda di destinazione. Chiaramente, per migliorare la trasparenza della posizione, è da preferirsi assegnare nomi alla coda che siano logici e indipendenti dalla posizione. L'utilizzo di nomi logici implica che ogni nome debba essere associato a un indirizzo di contatto, ad esempio una coppia (host, port), e che il mapping nome-indirizzo sia facilmente accessibile a un gestore code, come illustrato nella Figura 3. In realtà, un indirizzo di contatto contiene informazioni aggiuntive, incluso il protocollo da utilizzare, come TCP o UDP.

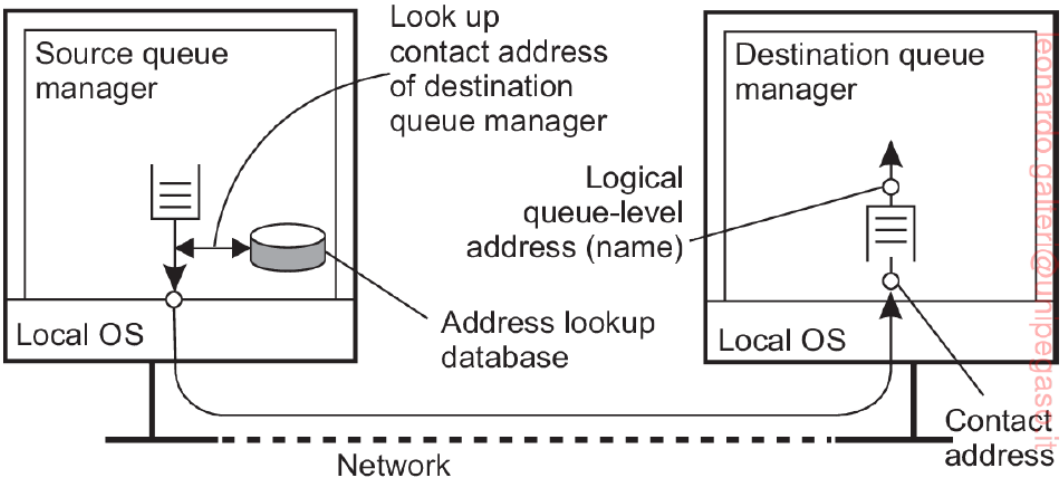


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Il modo in cui il mapping nome-indirizzo viene effettivamente reso disponibile a un gestore code è la seconda considerazione da fare. Un metodo comune consiste nell'implementare il mapping come tabella

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

di ricerca e distribuire copie di tale tabella a tutti i manager. Chiaramente, questo crea un problema di manutenzione, poiché molte, se non tutte, le tabelle devono essere aggiornate ogni volta che viene aggiunta o rinominata una nuova coda.

Questo ci porta al terzo problema, che ha a che fare con il mantenimento efficiente delle mappature nome-indirizzo. Se una coda di destinazione al gestore QM_B è nota al gestore code QM_A, QM_A può contattare QM_B direttamente per trasferire i messaggi. Ciò richiede essenzialmente che l'indirizzo di contatto di ciascun gestore della coda sia noto a tutti gli altri. Quando si ha a che fare con sistemi message-queuing estremamente grandi, i problemi di scalabilità sono inevitabili. In pratica, ci sono spesso gestori di code che funzionano anche come router, inoltrando i messaggi in arrivo ad altri gestori di code. Pertanto, un sistema message-queuing può gradualmente evolvere in una rete di overlay completa a livello di applicazione.

Se solo pochi router devono essere a conoscenza della topologia di rete, un gestore della coda di origine deve solo sapere a quale router adiacente, **R**, deve inoltrare un messaggio, data una coda di destinazione. Il router **R** potrebbe dover monitorare solo i router adiacenti per determinare dove inoltrare il messaggio e così via. Ovviamente, abbiamo ancora bisogno di mappature nome-indirizzo per tutti i gestori code, inclusi i router, ma non è difficile immaginare che queste tabelle potrebbero essere significativamente più piccole e più facili da gestire.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Broker di messaggi

L'integrazione di applicazioni esistenti e nuove in un unico sistema informativo distribuito coerente è un'area applicativa significativa dei sistemi message-queuing. Supponendo che la comunicazione con un'applicazione avvenga tramite messaggi, l'integrazione richiede che le applicazioni possano comprendere i messaggi che ricevono. In pratica, ciò richiede che i messaggi in uscita del mittente non solo aderiscano alla stessa semantica di quelli anticipati dal destinatario, ma abbiano anche lo stesso formato di quelli del destinatario.

Il mittente e il destinatario devono aderire allo stesso protocollo di messaggistica, parlando essenzialmente la stessa lingua. Il problema con questo approccio è che ogni volta che un'applicazione **A** con il proprio protocollo di messaggistica viene aggiunta al sistema, dovremo fornire i mezzi per convertire i rispettivi messaggi per ogni altra applicazione **B** che desidera comunicare con **A**. In un sistema con **N** applicazioni, saranno necessari convertitori di protocollo di messaggistica **N x N**.

Come per i protocolli di rete tradizionali, un'alternativa sarebbe quella di concordare un protocollo di messaggistica comune. Sfortunatamente, questa strategia è in genere inefficace per i sistemi message-queuing. Il problema è il livello operativo di astrazione di questi sistemi. Un protocollo di messaggistica comune ha senso solo se tutti i processi che lo utilizzano sono sufficientemente simili. Una soluzione unica per tutti non funzionerà se l'insieme di applicazioni che comprendono un sistema informativo distribuito è estremamente eterogenea, come del resto spesso accade.

Il grado di somiglianza può essere raggiunto aumentando il livello di astrazione, come con i messaggi XML, se consideriamo solo il formato e il significato dei messaggi.

In questo caso, i messaggi contengono informazioni sulla propria organizzazione e il modo in cui possono descrivere il loro contenuto è stato standardizzato. Di conseguenza, un'applicazione può fornire informazioni sulla struttura dei suoi messaggi che possono essere elaborati automaticamente. Ovviamente, queste informazioni sono raramente sufficienti; dobbiamo anche garantire che la semantica dei messaggi sia ben compresa.

Alla luce di questi problemi, l'approccio generale è quello di imparare ad accettare le differenze e di fornire il maggior numero possibile di semplici opzioni di conversione.

Nei sistemi message-queuing, le conversioni vengono gestite dai broker di messaggi, che sono nodi speciali in una rete di code. In un sistema message-queuing, un broker di messaggi funge da gateway a livello di applicazione. La sua funzione principale è quella di convertire i messaggi in arrivo in modo che l'applicazione ricevente possa comprenderli. Come illustrato nella Figura 4, un broker di messaggi è

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

semplicemente un'altra applicazione per un sistema message-queuing. In altre parole, un broker di messaggi non è in genere considerato come parte integrante del sistema message-queuing.

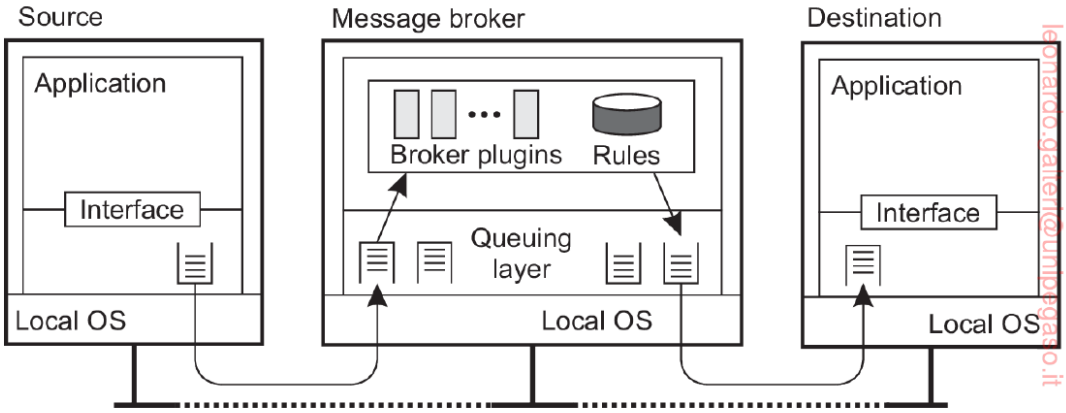


Figura 4 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Un broker di messaggi può essere considerata una semplice entità, come un formattatore di messaggi. Si supponga, ad esempio, che un messaggio in arrivo contenga una tabella da un database in cui i record sono separati da uno speciale delimitatore di fine record e i campi all'interno di un record hanno una lunghezza fissa. È possibile utilizzare un broker di messaggi per convertire i messaggi nel formato previsto dall'applicazione di destinazione se prevede un delimitatore diverso tra record e campi a lunghezza variabile.

In un ambiente più sofisticato, un broker di messaggi può fungere da gateway a livello di applicazione contenente informazioni codificate sui protocolli di messaggistica di più applicazioni. In generale, avremo un sottoprogramma separato per ogni coppia di applicazioni in grado di convertire i messaggi tra di loro. Questo sottoprogramma è rappresentato come un plug-in nella Figura 4 per sottolineare che tali componenti possono essere collegati o rimossi dinamicamente da un broker.

Un broker di messaggi viene spesso utilizzato per l'integrazione avanzata delle applicazioni aziendali (EAI). In questo caso, piuttosto che (semplicemente) convertire i messaggi, un broker è responsabile della corrispondenza delle applicazioni in base ai messaggi scambiati. In un'architettura di pubblicazione-sottoscrizione, le applicazioni inviano messaggi tramite pubblicazione. In particolare, possono inviare un messaggio sull'argomento **X** al broker dopo averlo pubblicato. Le applicazioni che hanno espresso interesse per i messaggi sull'argomento **X**, cioè si sono iscritte a questi messaggi, li riceveranno successivamente dal broker. Sono possibili anche forme più avanzate di mediazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Al centro di un broker di messaggi c'è un repository di regole per la conversione di un tipo di messaggio in un altro. Definire le regole e sviluppare i plugin è problematico. Anche se la maggior parte dei prodotti di messaggistica include sofisticati strumenti di sviluppo, il repository deve essere popolato da specialisti, persone esperte. I prodotti commerciali, ad esempio, sono spesso travisati come "intelligenza" quando, in realtà, solo il cervello degli esperti contiene intelligenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).